

MarketPilot AI — Study Notes

Beginner-friendly architecture and interview prep

Devansh · 2026

MarketPilot AI — Study Notes

These are your interview-ready notes. Read top to bottom the first time. After that, jump to any section.

Table of contents

1. What is MarketPilot AI? (one paragraph)
 2. The problem this solves
 3. Big picture — how the pieces fit
 4. Tech stack — what each tool is and why we chose it
 5. What is an AI agent? (the most important concept)
 6. The autonomy tiers — GREEN, YELLOW, RED
 7. Skills vs. Tools vs. Connectors
 8. The frontend — what's there
 9. The backend — what's there
 10. The 9 stores — our database (in memory)
 11. The API endpoints
 12. What we've built, step by step
 13. Glossary of every term
 14. Interview questions + good answers
 15. What's NOT built yet (be honest)
 16. How to run, test, demo
-

1. What is MarketPilot AI?

One sentence: It's a website you can log into where AI agents do real marketing work for your product — SEO audits, ad campaigns, copy drafts, competitor research — and ask you to approve the riskier ones before doing them.

Slightly longer: You connect your product (website URL, ad accounts, CMS, etc.), describe what your product is and who it's for, and then click a "Launch Agents" button. Specialist AI agents go off, **actually read your website, actually crawl competitor pages, actually draft copy, save findings**, and come back with a report PLUS a list of concrete actions you can approve with one click.

Not "AI shows you a chatbot." More like "AI is a marketing team in a tab."

2. The problem this solves

Small companies need a marketing team. They can't afford one. The old answer was "use a SaaS tool." But SaaS tools just **show you data**. They don't **do the work**. You still need a human to:

- Read the data
- Decide what to do
- Open the CMS and update the page
- Open Meta Ads and pause the bad ad set
- Write the email
- Send the email

MarketPilot AI is built so the agent does as much of that as it safely can, asks you only when the stakes are high, and **never** does anything destructive without a rollback path. That's the whole pitch.

3. Big picture — how the pieces fit

Imagine a kitchen.

- **The customer (you, Devansh, or a paying user)** orders food → that's the **frontend**: the buttons, screens, forms.
- **The waiter** carries the order to the kitchen → that's the **API**: the HTTP endpoints.
- **The head chef** (the AI agent) reads the order and decides what dishes to cook → that's the **agent loop** running on the backend.
- **The pantry** holds the ingredients: olive oil, salt, vegetables → those are the **tools** (web_search, crawl_site, audit_seo, write_draft).

- **The supplier trucks** bring the ingredients in from outside the kitchen — vegetables from a farm, oil from a factory → those are the **connectors** (the cheerio site connector that goes out to the public web; later: GA4 connector, GitHub connector, Meta Ads connector).
- **The recipe books** explain how to cook each dish → those are the **skills** (`SKILL.md` files for SEO Audit, Copywriting, Paid Ads, etc.).
- **The notebook the chef writes in** — "tonight I salted the steak, tomorrow I'll add pepper" → that's the **event log** + **stores** (every action recorded for later).
- **The "I need the customer's go-ahead before I serve the wasabi"** rule → that's the **tier gate** (GREEN = serve immediately, YELLOW = serve but say something, RED = ask the customer first).

That's everything. Now the rest of these notes just put names to those parts.

4. Tech stack — what each tool is and why we chose it

Frontend

Tool	What it is	Why we use it
Next.js (App Router)	A React framework. Lets you build a website with multiple pages, fast loading, server-side rendering.	Industry standard. Vercel made it. Has built-in routing, image optimization, and works with TypeScript out of the box.
React	A library for building user interfaces from small reusable pieces called "components" (like Lego blocks).	The most popular UI library in the world. Almost every modern web app uses it.
TypeScript	JavaScript + type safety. You write <code>let age: number = 5;</code> and TypeScript catches it if you accidentally try to do <code>age = "five"</code> .	Stops bugs before they happen. Auto-complete in the editor. Required for serious projects.
Tailwind CSS	A CSS framework where instead of writing custom CSS, you put utility classes right in your HTML: <code><div class="bg-blue-500 p-4 rounded-xl"></code> .	Fast to build with. Consistent look. No need to invent new class names. Easy to delete unused code.

Backend

Tool	What it is	Why we use it
Node.js	JavaScript that runs on the server (not in the browser).	Same language as the frontend. One mental model. Huge ecosystem.
Express	A thin HTTP framework for Node.js. You write <code>app.get("/health", (req, res) => res.json({ok:true}))</code> and you have an API.	Simple, mature, well-documented. Fine for our scale.
TypeScript	Same as frontend.	Same reason.
Google Gemini	Google's AI model (similar to OpenAI's GPT or Anthropic's Claude). We use Gemini 2.5 Flash (fast, cheap, good enough) and plan to add Gemini 2.5 Pro for harder planning tasks.	Has reliable "function calling" (the AI can decide to call our tools). Free tier is generous for development.
cheerio	A library that parses HTML and lets you query it like jQuery — <code>\$('#h1').text()</code> gives you the H1.	Lightweight, fast, doesn't require a real browser. Perfect for SEO crawling where we just want the page structure.
In-memory Maps	JavaScript's built-in key/value data structure. Like a dictionary.	We don't need a real database yet. When we do, we swap these out for SQLite, then Postgres.

What we don't use yet (and why we'll add them later)

- **Postgres / SQLite** — real databases. Will replace `Maps` when we need data to survive a restart.
- **Inngest / node-cron** — schedulers. Will add when we need "run SEO audit every Monday."
- **Clerk / Supabase Auth** — login/signup. Will add when there's more than one user.
- **Anthropic Claude SDK** — alternative LLM. We picked Gemini; could switch later.

The **framework spec** (`marketing-agent-framework.md`) lists all the "real production" tools. We picked simpler equivalents because we're solo and pre-launch. **This is a normal engineering tradeoff** and you should be able to defend it in an interview: *"We code to the interface, not the implementation. When the simpler tool stops fitting, the swap is small because the rest of the code doesn't know which tool is underneath."*

5. What is an AI agent? (the most important concept)

In plain English: an **AI agent** is when you give an AI model a **goal** and a **list of tools it can use**, and the AI decides — by itself, one step at a time — which tools to call until the goal is done.

What it is NOT

- It's not a chatbot. A chatbot says "here's an answer." An agent says "I should call tool A, then look at the result, then call tool B."
- It's not a single API call. It's a **loop** — call AI, AI says "use tool X", we run tool X, we send the result back to the AI, repeat.

How it works in our code

The loop lives in backend/src/lib/agent-loop.ts. Here's what happens in plain English:

1. **You click "Launch Agents"** in the frontend.
2. The frontend calls `POST /api/agent/start` with the campaign brief.
3. The backend creates a new "skill run" record (a database row).
4. The backend builds a long prompt: "You are MarketPilot AI. Here's the brief. Here are the tools you can use. Get the work done."
5. The backend sends that prompt to **Gemini** along with the list of tools.
6. Gemini responds with one of two things:
 - **"Call this tool with these arguments"** (e.g. `audit_seo("https://example.com")`)
 - **"I'm done. Here's the final report."** (it calls a special tool called `finish`)
7. If Gemini wants to call a tool:
 - **The tier gate runs first** (see next section).
 - If allowed, we run the tool. The tool returns a result.
 - We add the result to the conversation and send it back to Gemini.
8. Gemini sees the result and decides the next step. Repeat from step 6.
9. Loop ends when Gemini calls `finish` OR we hit a max of 12 tool calls (a safety cap).

The technical term for this is **ReAct loop** (**Reason** + **Act**). Most modern AI agents work this way.

Why this is powerful

Because each tool can do a small, reliable thing — fetch a URL, parse HTML, run a check — and the AI is the **glue** that chooses the order. You don't have to write the marketing logic; the AI does. You just have to write **safe, predictable tools**.

6. The autonomy tiers — GREEN, YELLOW, RED

This is the **single most important design idea** in MarketPilot AI. Memorize this.

The agent has access to many tools. Some are safe to run automatically. Some need a human sign-off. We use three lanes:

Tier	Color	What it means	Examples
GREEN		Pure reads. Run automatically. Don't bother the user.	<code>web_search</code> , <code>read_url</code> , <code>crawl_site</code> , <code>audit_seo</code> , <code>fetch_analytics</code>
YELLOW		Makes a small reversible change. Run automatically, but log it loudly so the user sees what happened and can undo.	<code>write_draft</code> , <code>update_meta_tag</code> , <code>publish_blog_post</code> , <code>pause_underperforming_ad</code>
RED		High stakes or hard to undo. Do not run . Create an approval record. Wait for a human to say yes.	<code>send_email_to_full_list</code> , <code>launch_ad_campaign</code> , <code>change_pricing</code> , <code>delete_page</code>

How it's enforced

Every tool declares its tier in `backend/src/lib/agent-tools.ts`:

```
export const toolMeta: Record<string, ToolMeta> = {
  web_search: { tier: "GREEN" },
  read_url: { tier: "GREEN" },
  crawl_site: { tier: "GREEN" },
  audit_seo: { tier: "GREEN" },
  write_draft: { tier: "YELLOW" },
};
```

Then a **single function** called the **tier gate** in `backend/src/lib/agent/tier-gate.ts` is called **before every tool dispatch**:

```
agent says "call tool X"
↓
tier-gate checks: what tier is X?
↓
GREEN → run it, log the decision, return result
YELLOW → run it, log a "notify" event, return result
RED → DON'T run it. Create an approval record. Tell the AI "blocked".
```

Two key rules

1. **Tier checks happen at the gate, not in the tool body.** The tool itself doesn't know about tiers. It assumes it's allowed. This means **forgetting the gate is impossible by accident**.
2. **Unknown tools default to RED.** If the agent asks for a tool we don't recognize, the gate blocks it. We call this **failing closed** — when in doubt, refuse.

Budget guards (separate idea, but related)

Even within a tier, there are **hard caps**:

- Max ad spend per day
- Max email recipients per send
- Max pages modified per day

If a call would exceed a cap, **it becomes RED regardless of its declared tier**. Money/blast-radius limits override autonomy.

7. Skills vs. Tools vs. Connectors

This is the second most important design idea. Three different things, often confused:

Concept	What it is	Where it lives	Example
Skill	A piece of marketing knowledge . A markdown document that says "this is how to run an SEO audit." Tells the AI how to think .	<code>.agents/skills/<skill-id>/SKILL.md</code>	<code>seo-audit/SKILL.md</code> : "First check the title, then headings, then schema..."
Tool	A verb . A small function the agent can actually call. Has a name, a typed input, a tier, and an implementation.	<code>backend/src/lib/agent-tools.ts</code>	<code>audit_seo(url)</code> → returns findings
Connector	A typed adapter that talks to one external service. Declares its capabilities.	<code>backend/src/lib/connectors/</code>	<code>cheerioSiteConnector</code> knows how to fetch and parse a webpage

The rule that ties them together

Skills never make HTTP calls. Tools never read other skills. Connectors never make decisions.

- Skills are pure knowledge. They tell the agent what to think about.
- Tools are pure verbs. They execute one action.
- Connectors are pure adapters. They talk to one outside service.

Why this separation matters

If you mix them up, you get a mess where you can't swap the LLM, you can't swap the database, and adding a new feature requires editing 10 files. Keeping them separate means:

- You can add a new skill (e.g. "TikTok content strategy") by just writing a new markdown file.
 - You can add a new tool (e.g. `pause_meta_ad`) by adding one function and one registry entry.
 - You can replace the cheerio connector with a Playwright connector (heavier, runs JS) by changing one file.
-

8. The frontend — what's there

All the UI currently lives in **one file**: `frontend/app/page.tsx` (~1670 lines). We'll split it into views/components in a later step.

Screens implemented (with mock data)

1. **Dashboard** — "Good evening, Devansh", stat cards, live agent run cards with progress bars, proposed actions, activity feed, recent drafts.
2. **Campaigns** (this is the **Workstreams** screen from the framework) — agent run detail. Shows a console-style trace of what the agent is doing (`tool_call` , `tool_result` , `thought` , `error`), plus stats, competitor keyword analysis, technical fixes, projected traffic chart.
3. **Product Profile** — form to describe your product: Identity (name, pitch, industry, stage), Narrative (long description), Brand DNA (voice, do's/don'ts, colors), KPIs, Ecosystem (the integrations panel — Website, GA4, Meta Ads, WordPress).

Screens NOT yet built (placeholders only)

4. **Agents** → will become **Chat**
5. **Drafts** → standalone drafts gallery
6. **Proposed Actions** → will become the **Approvals Inbox**
7. **SEO Reports** → will become the **Audits** history
8. **Integrations** → real OAuth flows
9. **Settings**

The design system

The whole UI uses a consistent palette:

- **Background**: slate-50 (very light gray) for pages, white for cards
- **Primary action**: indigo-600 (the buttons), often with a gradient to violet-600
- **Status colors**: emerald (success), amber (warning), rose (danger), slate (neutral)
- **Shape**: `rounded-2xl` cards with subtle borders, `rounded-xl` buttons

- **Spacing:** generous whitespace, `gap-4` between grid items
- **Motion:** pulsing dots for "live", soft progress bar animations, hover lifts

State management

Just **React's built-in** `useState`. No Redux, no Zustand. The framework spec says we don't need anything bigger until we have realtime multi-user state — which is far away.

How frontend talks to backend

The backend runs on `http://localhost:4000`. The frontend calls it with `fetch(...)`. Currently this is inline in `page.tsx`. The plan is to extract every call to `frontend/app/lib/api.ts` so we have one file that knows about every endpoint.

9. The backend — what's there

Folder structure (the parts that exist today)

```

backend/
├─ package.json
├─ tsconfig.json
├─ src/
│   ├─ server.ts           # Express bootstrap - CORS, /health, mounts routes
│   ├─ routes/
│   │   └─ agent.ts        # /api/agent/start, /api/agent/:id, /api/agent/:id/approve
│   └─ lib/
│       ├─ agent-loop.ts   # The agent loop (the ReAct loop from §5)
│       ├─ agent-tools.ts  # Tool declarations + tier registry + dispatch
│       └─ agent/
│           ├─ tier-gate.ts  # The gate function
│           ├─ index.ts     # barrel re-export
│           └─ __tests__/
│               └─ tier-gate.test.ts
│       ├─ connectors/
│           ├─ types.ts     # SiteConnector interface, CrawledPage type
│           └─ site/
│               └─ cheerio.ts # The HTML crawler/parser
│               └─ index.ts
│       └─ tools/
│           └─ seo/
│               ├─ audit-checks.ts # The ~10 SEO checks + scorer
│               └─ __tests__/
│                   └─ audit-seo.test.ts
│       └─ store/           # All 9 stores (next section)
│           ├─ types.ts
│           ├─ workspaces.ts
│           ├─ product-profile.ts
│           ├─ connections.ts
│           ├─ skill-runs.ts
│           ├─ tool-calls.ts
│           ├─ approvals.ts
│           ├─ audits.ts
│           ├─ performance.ts
│           ├─ events.ts
│           └─ index.ts

```

The flow of one request

When the frontend clicks "Launch Agents":

1. POST /api/agent/start → routes/agent.ts
2. create a skill run → store/skill-runs.ts
3. kick off the loop → lib/agent-loop.ts (runs in background)
4. return { taskId } → frontend
5. (background) loop calls Gemini, gets a tool call back
6. tier-gate checks tier → lib/agent/tier-gate.ts
7. gate writes event → store/events.ts
8. if allowed, dispatch → lib/agent-tools.ts → e.g. cheerioSiteConnector.crawl()
9. save audit → store/audits.ts
10. send result to Gemini, loop
11. Gemini calls "finish" → finalReport saved on the skill run

Meanwhile the frontend polls `GET /api/agent/:taskId` every 1.5 seconds to update the UI live.

10. The 9 stores — our database (in memory)

A **store** is just a file that holds one kind of data in a JavaScript `Map`, exposes a few methods (`create`, `get`, `list`, `update`), and hides the `Map` from the rest of the code.

We split data into 9 stores because the framework spec asks for them. Each one will become a database table later.

Store	Holds	Used by
workspaces.ts	The single default workspace (<code>ws_default</code>)	Every other store references it
product-profile.ts	The product description form (name, industry, voice, KPIs, etc.). One per workspace.	The agent reads this as memory before every run (planned). Frontend reads/writes via API (planned).
connections.ts	Site URL, GA4, GSC, Meta Ads, WordPress, etc.	Connectors look up their config here.
skill-runs.ts	Every agent run — its status, steps, drafts, final report. This is the one currently doing real work.	The agent loop writes to it; the frontend polls it.
tool-calls.ts	One record per tool invocation, with rollback payload.	Currently empty — will be filled when we add real write tools.
approvals.ts	First-class approval records (RED-tier blocked calls).	Tier gate creates these on RED. Inbox UI (Step 4) reads them.
audits.ts	Persistent SEO/CRO/ads audits. Step 3 fills this.	<code>audit_seo</code> writes; <code>/api/audits</code> reads.
performance.ts	Daily snapshots: traffic, MRR, CAC, churn.	Currently empty. Scheduler fills it daily (Step 6+).
events.ts	Append-only log of every meaningful event.	Tier gate writes here. <code>/api/events</code> reads.

Why this split is interview-worthy

If an interviewer asks "**why split it instead of one big store?**", here's the answer:

1. **Each store has one job**, so changes are local. Changing how audits work doesn't touch how skill runs work.
2. **Each store maps directly to a future database table**. When we switch to SQLite, each store becomes a table with the same fields. The rest of the code doesn't change.
3. **Each store has a stable interface** (`create` , `get` , `list`). The implementation (Map vs SQLite vs Postgres) is hidden.
4. This is called **separation of concerns** and **the repository pattern**. Both are textbook software design principles.

11. The API endpoints

This is the contract between frontend and backend. All endpoints live in `backend/src/server.ts` and `backend/src/routes/agent.ts`.

Method	Path	Purpose	Used by
GET	<code>/health</code>	Sanity check — is the server alive?	Anyone, deploy probes
POST	<code>/api/agent/start</code>	Start a new agent run. Body: <code>skillId</code> + campaign brief. Returns <code>{ taskId }</code> .	Frontend "Launch Agents" button
GET	<code>/api/agent/:taskId</code>	Poll status of a run.	Frontend polls this every 1.5–3 seconds
POST	<code>/api/agent/:taskId/approve</code>	Approve/reject a proposed action. Body: <code>{ actionId, approved }</code> .	Frontend Approve/Reject buttons
GET	<code>/api/events</code>	Read the tail of the event log (every tool call, gate decision, etc.).	Devs (for now); future Activity feed
GET	<code>/api/audits</code>	List all SEO audits.	Devs (for now); future Audits screen
GET	<code>/api/audits/:id</code>	Drill into one audit.	Devs (for now); future Audits screen

Planned endpoints (will appear in future steps)

`GET/PUT /api/profile`, `GET/POST /api/connections`, `GET /api/approvals`, `POST /api/approvals/:id/decide`, `GET /api/skill-runs`, `POST /api/chat`.

12. What we've built, step by step

This is the historical narrative — useful for an interviewer asking *"walk me through your work."*

Phase 0 — Initial UI prototype (before structured steps)

- Built a single-page Next.js dashboard with mock data.
- Got familiar with Tailwind + the SaaS dashboard style.
- Connected sidebar → main view rendering pattern.
- Had a working but limited Express backend that called Gemini in a basic loop.

Step 1 — Storage restructure

Goal: Split the single `agent-store.ts` into 9 typed stores. No behavior change.

Why: Future features (tier system, connectors, approvals UI, scheduler) need clean seams to write to the right place. One tangled `Map` would force them to all touch the same file.

What we did:

- Created `backend/src/lib/store/types.ts` with every shared type (`Workspace` , `ProductProfile` , `Connection` , `SkillRun` , `ToolCall` , `Approval` , `Audit` , `PerformanceSnapshot` , `AgentEvent`).
- Created 9 store files, each with its own `Map` and CRUD methods.
- Migrated all logic from the old `agent-store.ts` into `store/skill-runs.ts` .
- Updated import sites in `agent-loop.ts` , `agent-tools.ts` , `routes/agent.ts` .
- Deleted the old file.
- Type-check clean. All endpoints return identical JSON.

Interview soundbite: "I refactored the single-file in-memory store into 9 typed stores using the repository pattern, so each domain entity has a clean seam. JSON wire format stayed byte-compatible — frontend didn't notice the rename."

Step 2 — Tier system

Goal: Add the GREEN/YELLOW/RED gate that runs before every tool dispatch.

Why: Framework rule §5.2 — "tier checks happen at the gate, not in the tool body." This is the central safety mechanism.

What we did:

- Added a `toolMeta` registry in `agent-tools.ts` mapping each tool to its tier.
- Created `lib/agent/tier-gate.ts` — a pure function `tierGate({ skillRunId, toolName, input }) → { kind, tier, ... }` .
- Wired the gate into `agent-loop.ts` — every tool call now goes through it.
- Unknown tools default to RED (**fail closed**).
- Every gate decision writes a `tool_gated` event to the events store.
- RED-tier calls create an approval record and tell the AI "blocked, work around it or finish."
- Wrote a unit test covering all 3 branches + the events written.
- Exposed `GET /api/events` so we can verify the gate is firing.

Interview soundbite: "I implemented a three-tier autonomy model — green auto-executes, yellow auto-executes + notifies, red gets parked as an approval. The gate is a single function called before every tool dispatch and writes one event per decision. Unknown tools fail closed."

Step 3 — First real connector + SEO audit

Goal: Build the first connector and let the agent do **real structured work on a real URL**.

Why: Before this step, the agent only had `web_search` (vague DuckDuckGo) and `read_url` (raw HTML strip). Neither produced structured findings. This step changes that.

What we did:

- Installed `cheerio` — a lightweight HTML parser.
- Created `lib/connectors/types.ts` — the `SiteConnector` interface every site connector must satisfy. Connectors declare **capabilities** (`canCrawl`, `canWriteMeta`, etc.).
- Created `lib/connectors/site/cheerio.ts` — fetches a URL, parses with cheerio, returns a typed `CrawledPage` (title, meta, headings, alt-text gaps, link counts, schema, canonical, language, viewport).
- Created `lib/tools/seo/audit-checks.ts` — a **pure function** `runAuditChecks(page)` that runs ~10 SEO checks and returns findings tagged by severity. Also a `scoreFromFindings` that produces a 0–100 health score.
- Added two new tools (`crawl_site`, `audit_seo`) to `agent-tools.ts`, both GREEN tier.
- `audit_seo` saves a real audit record to the `audits` store.
- Exposed `GET /api/audits` and `GET /api/audits/:id`.
- Wrote a unit test that runs against `https://example.com` and asserts the full chain works.
- End-to-end live test: started a run, agent called `audit_seo`, audit landed in the store with score 65, 6 findings.

Interview soundbite: "I built the first real connector — a cheerio-based site crawler that returns structured SEO data — and wired it through two new tools, `crawl_site` and `audit_seo`. The audit checks are a pure function over the crawled page, so they're easy to unit-test. Findings get persisted to the audits store and exposed via a read-only endpoint."

Steps still to do

- **Step 4** — Approvals inbox UI (the first **visible** payoff of the tier gate).
- **Step 5** — First YELLOW write tool: `add_alt_text` via a GitHub-MDX connector (PR-based rollback built in).
- **Step 6** — Scheduler (node-cron) + memory wiring (agent reads product profile before every run).
- **Step 7** — Wider tool coverage (ads, email, content) + missing UI screens (Audits, Chat).

13. Glossary of every term

Agent — An LLM + a list of tools + a loop. Picks the next tool to call, repeats until done.

Agent loop — The repeating cycle: LLM picks a tool → we run it → we send the result back to the LLM → repeat. Lives in `agent-loop.ts`.

Approval — A record that says "the agent wanted to do this RED-tier thing — please decide."
Created by the tier gate, decided by the user.

Audit — A saved SEO/CRO/ads report. Has scope (what was looked at), findings (issues found), and a score. Lives in the `audits` store.

Budget guard — A hard cap that overrides tiers. E.g. "no more than \$50/day in ad spend." Even an approved action can't exceed it.

Capability — A flag on a connector declaring what it can do (`canCrawl`, `canWriteMeta`, etc.).
Tools check capabilities before writing.

Cheerio — A library that parses HTML in Node.js (no browser needed) and lets you query it with jQuery-style selectors.

Connector — A typed adapter that talks to ONE external service. Hides the messy details of HTTP, auth, OAuth.

Cron — A way to run jobs on a schedule. `node-cron` lets us say "every Monday at 6am, run this function."

CRUD — Create, Read, Update, Delete. The four basic operations on data. Our stores expose CRUD methods.

Endpoint — A URL on the backend that does something. `POST /api/agent/start` is an endpoint.

Event log — An append-only list of every meaningful thing that happened. Tier gate decisions, tool calls, approvals — all logged.

Express — The thin web framework on the backend. Handles HTTP requests.

Failing closed — Default to "no" when uncertain. Unknown tool → RED. Unknown user → reject.
The safe direction.

Framework spec — The big architecture doc (`marketing-agent-framework.md`). The target we're building toward.

Gemini — Google's family of AI models. We use Gemini 2.5 Flash.

JSON-LD — JavaScript Object Notation for Linked Data. Structured data Google reads to understand your page. SEO loves it.

LLM — Large Language Model. Gemini, Claude, GPT — all LLMs.

Map (JavaScript) — Built-in key/value store. Like a Python dict.

Next.js — The React framework powering the frontend.

ReAct — Reason + Act. The pattern of "AI reasons about next step, takes an action, looks at result, reasons again." Our loop is ReAct.

Repository pattern — A design pattern where data access goes through a single interface (the "repository") that hides the underlying database. Our stores follow this.

Rollback — The ability to undo a write. Every YELLOW tool must implement it. RED tools that can't be rolled back stay RED forever.

Schema markup — JSON-LD blocks on a webpage that tell Google "this is a product, this is a blog post, this is a review." Boosts SEO.

Separation of concerns — A design principle: each module does one job. Skills know things, tools do things, connectors talk to services.

Skill — A markdown file that describes one marketing discipline (SEO Audit, Copywriting). The AI reads it as a recipe.

Skill run — One execution of one skill on one campaign brief. Has a status, steps, drafts, final report.

Store — Our word for "database table." Currently in-memory **Maps**; will become SQLite/Postgres tables later.

Tailwind — A CSS framework where you compose styles from utility classes in your HTML.

Tier — GREEN, YELLOW, or RED — the autonomy level of a tool. Determines whether it runs automatically.

Tier gate — The function that checks every tool call against its tier and decides what to do.

Tool — A function the agent can call. Each tool has a name, typed input, tier, and implementation.

Tool calling (also "function calling") — The LLM feature where instead of replying in text, the model says "call function X with these arguments." Gemini, Claude, and GPT all support this.

TypeScript — JavaScript with type annotations. Catches bugs at compile time.

Verify — Re-fetching state after a write to confirm the change landed. Required for every YELLOW/RED tool by the framework.

Workspace — A single tenant. Today we have one default. Multi-tenant comes later.

Zod — A TypeScript library for validating data shapes. We'll use it for tool input validation in Step 4+.

14. Interview questions + good answers

Q: Walk me through the architecture.

A: Three layers. The frontend (Next.js + React + Tailwind) shows the user dashboard, campaign briefs, and run results. The backend (Node + Express + TypeScript) runs an agent loop: it loads memory about the workspace, sends a prompt with available tools to Google Gemini, gets back a tool call, runs that tool through a tier gate, sends the result back, and repeats until the agent calls `finish`. Behind the agent are three boxes: stores (data, currently in-memory), tools (verbs the agent can call), and connectors (typed adapters to outside services like the cheerio-based site crawler).

Q: What is an AI agent?

A: An LLM plus a list of tools plus a loop. The LLM reasons about the next step, picks a tool, the tool runs, the result comes back, the LLM reasons again. Stops when the LLM declares done or we hit a safety cap. Our agent uses Gemini 2.5 Flash with function calling.

Q: Why split the storage into 9 separate stores?

A: Single responsibility. Each store maps to one future database table. Each has a stable interface (`create` , `get` , `list`) so swapping the implementation — Map today, SQLite tomorrow, Postgres later — doesn't ripple through the rest of the code. This is the repository pattern.

Q: What is the tier system and why does it exist?

A: Three lanes — GREEN auto-executes (reads), YELLOW auto-executes with a notify log (reversible writes), RED creates an approval and waits for a human. Every tool declares its tier. The gate is a single function called before every dispatch. Unknown tools default to RED — fail closed. This means the agent can do real work autonomously without a human having to baby-sit every read, but anything risky still gets gated.

Q: How does the agent know which tools to call?

A: We send Gemini a list of tool declarations — name, description, JSON schema for the input. Gemini's response either contains a `functionCall` part (it wants to use a tool) or just text. If it's a function call, we run the matching dispatch case in our backend.

Q: How do you prevent the agent from running a tool that doesn't exist?

A: The tier gate fails closed. If a tool name isn't in our `toolMeta` registry, the gate treats it as RED and creates an approval instead of running. We also dispatch via an exhaustive switch — unknown tool names return `"Unknown tool: X"`.

Q: What is a connector?

A: A typed adapter that wraps one external service. Our `cheerioSiteConnector` knows how to fetch a URL and return a structured `CrawledPage`. Connectors declare capabilities — e.g. cheerio has `canCrawl: true`, `canWriteMeta: false`. Tools check those capabilities before attempting writes. Future connectors (WordPress, GitHub-MDX, GA4) implement the same interface, so tools written today work against any of them.

Q: What is the event log?

A: An append-only list of every meaningful event — tool calls, gate decisions, approvals, rollbacks, LLM calls. Used for the activity feed and for debugging. The tier gate writes one event per decision. Routes never `console.log` business data — they emit events.

Q: Why use Gemini instead of OpenAI or Claude?

A: Engineering tradeoff. Generous free tier for development, reliable function calling, fast inference with 2.5 Flash. The code is structured so swapping the LLM is a single-file change inside `agent/`. The framework spec actually calls for Claude — we picked Gemini and documented the deviation.

Q: Why in-memory storage?

A: Solo project, pre-launch. Real databases (Postgres, SQLite) add operational complexity we don't need yet. Each store has a stable interface, so swapping `Map` for `better-sqlite3` later is a one-file change per store. Premature optimization is a real cost.

Q: How do you test the SEO audit logic?

A: Two layers. First, `audit-checks.ts` is a pure function — input is a `CrawledPage` object, output is a list of findings. No I/O. Easy to unit-test with fake inputs. Second, the connector itself is tested against a live URL (`example.com`) — slower but verifies the real fetch + parse chain. Both tests are runnable with `npm run test`.

Q: What's the difference between a skill and a tool?

A: A skill is knowledge — a markdown document that tells the AI how to think about a problem ("for SEO audit: check title, then meta, then headings..."). A tool is a verb — a typed function the AI can call (`audit_seo(url)`). Skills never make HTTP calls; only tools do.

Q: How would you scale this to 100 customers?

A: Three changes. (1) Replace in-memory stores with Postgres — same interfaces, real persistence. (2) Add a scheduler like Inngest so weekly cron jobs run reliably even if the server restarts. (3) Add auth (Clerk or Supabase Auth) so each workspace's data is isolated. The agent loop and tool system don't change.

Q: What's the hardest design decision you faced?

A: Where to draw the line between automation and approval. Going too automatic — agent ships a typo to your homepage. Going too gated — every tiny read needs a click, defeats the point. The three-tier system was the answer: pure reads run silently, reversible writes auto-run but log loudly, irreversible writes wait. Plus budget guards as hard caps for money/blast-radius.

Q: What would you build next if you had a week?

A: The approvals inbox UI (Step 4). The tier gate already creates approval records for RED calls — they're sitting in the store with no screen to show them. Building that screen is the first time a human user would actually see the autonomy system in action. After that, the first YELLOW write tool — `add_alt_text` via a GitHub-MDX connector, where rollback is "revert the commit."

15. What's NOT built yet (be honest)

If you're asked "what's missing," here's the truth:

- **No real database.** Restart the backend and everything resets. Audits, runs, approvals — all gone. Intentional for dev.
- **No auth.** Single user, single workspace.
- **No real OAuth.** The "Ecosystem" card on Product Profile shows fake "Connect" buttons. Nothing actually connects.
- **No scheduler.** "Run an SEO audit every Monday" doesn't work yet.
- **Frontend isn't wired to the backend for most screens.** Dashboard / Campaigns / Product Profile show mock data. Only the **Launch Agents** flow actually hits the backend.
- **Approvals inbox screen doesn't exist.** RED approvals are in the store but no UI to decide them.
- **The frontend's "Live trace" on the Campaigns screen is mock data, not the real `steps` array.**
- **No write tools yet.** The agent can read the web and save drafts, but it can't actually publish a blog post, send an email, or pause an ad.
- **No rollback or verify implementations.** When write tools land, those will too.
- **The 41 skill files in `.agents/skills/` are loaded but the agent currently gets a generic prompt — no per-skill tool filtering yet.**

These are all known and slotted into the build plan (Steps 4–7).

16. How to run, test, demo

Run the frontend

```
cd c:\Users\devan\OneDrive\Desktop\marketing-agent-platform\frontend
npm run dev
```

Opens at `http://localhost:3000`. Click between Dashboard, Campaigns, and Product Profile in the sidebar.

Run the backend

```
cd c:\Users\devan\OneDrive\Desktop\marketing-agent-platform\backend
npm run dev
```

Listens on `http://localhost:4000` . Sanity check: `curl http://localhost:4000/health` .

Run unit tests (no Gemini, no server)

```
cd c:\Users\devan\OneDrive\Desktop\marketing-agent-platform\backend

# Step 2 – tier gate
npx tsx src/lib/agent/__tests__/tier-gate.test.ts

# Step 3 – cheerio crawl + audit against example.com
npx tsx src/lib/tools/seo/__tests__/audit-seo.test.ts
```

Both should print a list of `ok:` lines and end with "All tests passed."

Run a real agent end-to-end (needs `GEMINI_API_KEY` in `backend/.env`)

PowerShell:

```
# Start the backend first (npm run dev), then in another shell:
$resp = Invoke-RestMethod -Method Post -Uri "http://localhost:4000/api/agent/start" `
  -ContentType "application/json" `
  -Body (@{
    skillId = "seo-audit"
    productName = "Demo"
    targetAudience = "anyone"
    campaignGoal = "Audit https://example.com using audit_seo."
    brandTone = "Professional"
    mainChannel = "Website SEO"
    campaignBudget = "Low"
    launchTimeline = "now"
  } | ConvertTo-Json)
$resp
Start-Sleep -Seconds 30

# Inspect what happened:
Invoke-RestMethod "http://localhost:4000/api/events" | ConvertTo-Json -Depth 5
Invoke-RestMethod "http://localhost:4000/api/audits" | ConvertTo-Json -Depth 8
Invoke-RestMethod "http://localhost:4000/api/agent/ $($resp.taskId)" | ConvertTo-Json -Depth 8
```

Type-check anytime

```
cd backend; npx tsc --noEmit
cd ../frontend; npx tsc --noEmit
```

Should exit 0 with no output.

End of notes

If anything here is unclear, write down the term, then re-read its section. Then look at the file the section links to. Reading the actual code is the final step — by the end of these notes you should already know what's in each file before you open it.

Last updated after Step 3 — first connector + SEO audit live.